# ПП

# Разбор `main.cpp` и `test_bank.cpp` — с пояснениями на каждую строку

## Часть 1. Теория по `main.cpp`

### Что вообще делает `main.cpp`

**Это не «библиотека», а приложение.** У него одна задача: **сымитировать экономику N секунд** и **в конце проверить два инварианта**. Всё остальное — вспомогательное.

Структура `main.cpp` сверху вниз:

1. `#include` — заголовки.
2. Анонимный `namespace` — внутренние глобальные функции и переменные (не видны снаружи).
3. `struct Args` — структура для хранения аргументов командной строки.
4. `parse_positive_int` — строгий парсер положительного целого.
5. `parse_args` — разбор `argc/argv` в `Args`.
6. `main` — сама симуляция.

Разберём теорию по частям.

---

### 1. Что такое `namespace {}` (анонимный `namespace`)

```
namespace {
    std::atomic<bool> g_stop{false};
    thread_local std::mt19937 g_rng{std::random_device{}()};
    void signal_handler(int) { g_stop.store(true); }
    int rand_int(int n) { ... }
```

```
    void log(const std::string& s) { ... }
}
```

**Анонимный** `namespace` — это способ сказать компилятору: «всё, что здесь объявлено, **видно только в этом** `.cpp`**-файле**».

Это **внутренняя линковка** (`internal linkage`). Если бы два разных `.cpp` объявили функцию `log` в анонимном `namespace`, это были бы **две разные** `log`, они не столкнулись бы на этапе линковки.

**Зачем это нужно:**

- **Никаких конфликтов имён** при линковке (в большом проекте много «log», «rand_int»).
- **Никто снаружи не может вызвать** эти функции — они приватны для модуля.
- Альтернатива — `static` перед каждым объявлением. Анонимный `namespace` — современный способ, покрывает всё разом.

**Важно:** `main.cpp` и `test_bank.cpp` — **два разных исполняемых файла**. Даже если бы `log` не был в `namespace`, конфликта не было бы. Но правило хорошего тона — **все вспомогательные функции — в анонимный namespace**.

---

## 2. `std::atomic<bool> g_stop` — почему `atomic`

```
std::atomic<bool> g_stop{false};
```

**Что это:** глобальная переменная «пора завершаться». Её пишет `signal_handler`, читают все потоки.

**Почему** `atomic`**, а не** `bool`**:**

- **Из signal handler в глобальную переменную писать опасно.** Поток может быть вытеснен в середине записи, если это неатомарная операция.
- **Из разных потоков читать одновременно тоже опасно** — data race → UB.
- `atomic` гарантирует: запись и чтение **неделимы**, гонки нет.

Плюс `bool` в C++ не гарантирует атомарность записи. На x86 `bool` скорее всего будет атомарным «случайно», но на ARM — нет. `std::atomic<bool>` — **портативно и корректно**.

---

## 3. `thread_local std::mt19937 g_rng`

```
thread_local std::mt19937 g_rng{std::random_device{}()};
```

`thread_local` — у **каждого потока** свой экземпляр переменной.

**Зачем:**

- Если бы `g_rng` был обычной глобальной — **все потоки конкурировали бы** за один генератор, а `std::mt19937` **не потокобезопасен** → data race → UB.
- С `thread_local` у каждого потока **свой** генератор. Никаких гонок.

`std::random_device{}()` — сид (затравка). `std::random_device` берёт **истинную случайность** из ОС (энтропию). Каждый поток получает **разный** сид → **разные** последовательности случайных чисел.

`std::mt19937` — быстрый PRNG (Mersenne Twister). Не криптостойкий, но для симуляции — идеально.

**Итого:** каждый поток кидает «свои» случайные числа, не мешая другим.

---

## 4. `void signal_handler(int)`

```
void signal_handler(int) { g_stop.store(true); }
```

**Обработчик Ctrl+C.** Регистрируется через `std::signal(SIGINT, signal_handler)` в `main`.

**Что можно и нельзя в signal handler:**

- **МОЖНО:** атомарные операции (`store`, `load`), `volatile sig_atomic_t`.
- **НЕЛЬЗЯ:** мьютексы (`lock`), `cout`, `malloc` / `new`, любые функции, которые могут спать, `throw`.

**Причина:** handler вызывается **асинхронно** — в любой момент, даже в середине `malloc` или `cout`. Если handler попробует взять мьютекс, который держит основной поток, — deadlock. Если попробует `cout`, когда `cout` в середине вывода другого потока — UB.

Поэтому — **только** `g_stop.store(true)`. Всё остальное делают сами потоки: они **проверяют** флаг и завершают цикл.

---

## 5. `int rand_int(int n)`

```
int rand_int(int n) {
    return std::uniform_int_distribution<int>(0, n - 1)(g_rn
g);
}
```

**Что делает:** возвращает случайное `int` в `[0, n-1]`. `rand_int(3)` → 0, 1 или 2.

`std::uniform_int_distribution` — равномерное распределение. Не `rand() % n` (там перекос из-за модуля), а **честное** равномерное.

**Почему** `g_rng` — свой генератор на поток (см. выше).

---

## 6. `void log(const std::string& s)`

```
void log(const std::string& s) {
    std::osyncstream(std::cout) << s << "\n";
}
```

**Что делает:** печатает строку в `cout` **атомарно**.

**Проблема** `std::cout`**:** он потокобезопасен (`operator<<` не рвёт данные), но **вывод двух потоков перемешивается**. Например, один печатает `"Director pays salary 100"`, другой — `"Mother buys 40"`. На экране получится `"Director Mother pays buys 40 salary 100"` — куски перемешаны.

`std::osyncstream` — обёртка над `cout`, которая **буферизует всё**, что в неё записали, и **сливает в** `cout` **одним куском** при разрушении (в конце выражения). Каждая строка выходит **целиком**.

`const std::string&` — передаём по const-ссылке, чтобы не копировать строку.

## 7. `struct Args` — почему не класс и что внутри

```
struct Args {
    std::string mode = "parallel";
    int seconds = 5;
    bool quiet = false;
};
```

`struct` — **потому что все поля публичные**, никакой инкапсуляции не надо. `struct` в C++ — это `class` с `public:` по умолчанию. Для «мешка с настройками» — идеально.

**Начальные значения** (`= "parallel"`, `= 5`, `= false`) — **дефолты**. Если пользователь не указал флаг — берётся дефолт.

**Почему не `class`:** у `class` по умолчанию всё `private`, пришлось бы писать `public:` — лишний шум. И «настройки» — это **данные**, не поведение.

## 8. `parse_positive_int` — зачем такие сложности

```
static int parse_positive_int(const char* s, const char* arg_
name) {
    size_t pos = 0;
    int val = 0;
    try {
        val = std::stoi(s, &pos);
    } catch (const std::exception&) {
        throw std::runtime_error(std::string(arg_name) + " mu
st be an integer");
    }
    if (pos != std::strlen(s))
        throw std::runtime_error(std::string(arg_name) + " mu
st be an integer");
    if (val <= 0)
```

```
        throw std::runtime_error(std::string(arg_name) + " mu
st be positive");
    return val;
}
```

**Зачем не просто `std::stoi(s)`:**

`std::stoi` **молча читает префикс** и **не жалуется**, если после числа идёт мусор. `std::stoi("1hello")` вернёт `1` и **не бросит исключение** — это известная ловушка.

Второй аргумент `&pos` — **сюда** `stoi` кладёт **позицию**, до которой прочитал. Проверяем: если `pos == strlen(s)`, значит прочитали **всю строку**. Если `pos < strlen(s)`, в строке был **хвост** — ошибка.

**Также** ловим случай `"abc"` — `stoi` бросает `invalid_argument`. Ловим **любое** исключение `std::exception` (вдруг `out_of_range` при огромном числе).

**Три проверки в итоге:**

1. `stoi` не бросил (иначе ловим).
2. `pos` совпадает с длиной (нет хвоста).
3. Значение положительное (`> 0`).

`static` — внутренняя линковка, то же, что анонимный `namespace`, но короче для одной функции.

---

## 9. `parse_args` — как разбирается `argc/argv`

```
static Args parse_args(int argc, char** argv) {
    Args a;
    for (int i = 1; i < argc; ++i) {
        std::string s = argv[i];
        if (s == "--quiet") a.quiet = true;
        else if (s == "--mode" && i + 1 < argc) a.mode = argv
[++i];
        else if (s == "--seconds" && i + 1 < argc) a.seconds
= parse_positive_int(argv[++i], "--seconds");
```

```
        else throw std::runtime_error("unknown argument: " +
s);
    }
    if (a.mode != "single" && a.mode != "parallel")
        throw std::runtime_error("--mode must be single or pa
rallel");
    return a;
}
```

**Что такое `argc` и `argv`:**

- `argc` = `argument count` — **сколько** аргументов (включая имя программы).
- `argv` = `argument vector` — **массив строк** (указателей на `char`).
- `argv[0]` — **имя программы** (`"bank.exe"`). Поэтому цикл **с i = 1**, а не с 0.

**Разбор по шагам:**

1. `for (int i = 1; i < argc; ++i)` — идём по всем аргументам, кроме имени.
2. `std::string s = argv[i];` — копируем `char*` в `std::string` для удобства (сравнение `==`).
3. `if (s == "--quiet")` — флаг без значения.
4. `else if (s == "--mode" && i + 1 < argc)` — флаг **со значением**. Проверяем `i + 1 < argc` — вдруг после `-mode` ничего нет.
5. `argv[++i]` — **сначала увеличиваем** `i`, потом берём `argv[i]`. Так мы «съедаем» следующий аргумент — это значение флага.
6. `else throw` — неизвестный флаг → ошибка.
7. После цикла — **проверка значения** `-mode`. `parse_positive_int` для `-seconds` уже проверила, что оно положительное.

**Пример:**

```
bank --mode parallel --seconds 10 --quiet
```

- `argc == 5`

- `argv = ["bank", "--mode", "parallel", "--seconds", "10", "--quiet"]`
- Цикл:
  - `i=1`: `s="--mode"`, `i+1<5` → `a.mode = argv[2] = "parallel"`, `i` стал 2 (пропустили "parallel").
  - `i=3`: `s="--seconds"`, `i+1<5` → `a.seconds = 10`, `i` стал 4.
  - `i=5`: `i < argc` = `5 < 6`? Wait — я напутал, `argc` для 6 элементов (имя + 5) — **6**. Пересчёт: `argv = ["bank"(0), "--mode"(1), "parallel"(2), "--seconds"(3), "10"(4), "--quiet"(5)]`, `argc=6`.
  - `i=1`: `-mode`, съедаем `argv[2]="parallel"`, `i` стал 2.
  - `i=3`: `-seconds`, съедаем `argv[4]="10"`, `i` стал 4.
  - `i=5`: `-quiet`, `a.quiet = true`.
  - `i=6`: выход.

**Итог:** `Args{mode="parallel", seconds=10, quiet=true}`.

---

# 10. `main` — по блокам

## 10.1. Разбор аргументов

```
Args args;
try { args = parse_args(argc, argv); }
catch (const std::exception& e) {
    std::cerr << "usage: ...\n";
    std::cerr << "error: " << e.what() << "\n";
    return 1;
}
```

Если парсинг упал — печатаем usage и ошибку, выходим с кодом 1. Требование п.5: «некорректные аргументы → понятное сообщение и ненулевой код возврата».

## 10.2. Ctrl+C

```
std::signal(SIGINT, signal_handler);
```

`SIGINT` — прерывание с клавиатуры (Ctrl+C). `signal_handler` — функция, которую вызовет ОС.

## 10.3. Начальное состояние

```
constexpr Money TOTAL = 12900;
Bank bank(TOTAL);
AccountId directorAcc = bank.OpenAccount();
AccountId fatherAcc   = bank.OpenAccount();
AccountId sellerAcc   = bank.OpenAccount();
```

- `TOTAL` — вся сумма в системе.
- `Bank(TOTAL)` — банк со всеми наличными в обороте (счета пустые).
- Три счёта: Директор, Отец, Продавец.

```
Wallet directorWallet(10000);
Wallet fatherWallet(500);
Wallet motherWallet(500);
Wallet child1Wallet(200);
Wallet child2Wallet(200);
Wallet sellerWallet(1500);
```

Шесть кошельков. **Сумма = 12900 = TOTAL**. Это важно для инвариантов.

## 10.4. Акторы

```
struct Actor {
    std::string name;
    std::function<void()> act;
};
std::vector<Actor> actors;
```

`Actor` — «участник» с именем и лямбдой-действием. `std::function<void()>` — обёртка над **любой вызываемой сущностью без аргументов и без возврата** (лямбда, функция, функтор).

**Зачем `std::function`, а не напрямую лямбды:** лямбды в `vector` имеют **разные типы** (у каждой свой unique type), их нельзя хранить в одном `vector`. `std::function` стирает типы — можно хранить все лямбды **в одном векторе**.

## 10.5. Цикл симуляции

```
auto deadline = std::chrono::steady_clock::now() + std::chron
o::seconds(args.seconds);
auto t0 = std::chrono::steady_clock::now();
auto opsBefore = bank.GetOperationsCount();
```

- `deadline` — момент, когда пора заканчивать.
- `t0` — начало замера (для `steady_clock`, только для подсчёта `elapsed`).
- `opsBefore` — счётчик «до», чтобы потом вычесть.

## 10.6. `single` режим

```
while (!g_stop.load() && std::chrono::steady_clock::now() < d
eadline) {
    for (auto& actor : actors) {
        actor.act();
        if (g_stop.load()) break;
    }
}
```

- Внешний `while`: пока не Ctrl+C и не истёк дедлайн.
- Внутри — обходим всех акторов по очереди, каждый делает один ход.
- После каждого хода проверяем `g_stop` — вдруг уже нажали Ctrl+C.
- **Один поток**, никакой параллельности.

## 10.7. `parallel` режим

```
std::vector<std::jthread> threads;
for (auto& actor : actors) {
    threads.emplace_back([&actor, &deadline] {
        while (!g_stop.load() && std::chrono::steady_clock::n
ow() < deadline) {
            actor.act();
        }
    });
}
while (!g_stop.load() && std::chrono::steady_clock::now() < d
eadline) {
    std::this_thread::sleep_for(std::chrono::milliseconds(2
0));
}
g_stop.store(true);
```

- `std::vector<std::jthread>` — вектор потоков. `jthread` сам `join`ится в деструкторе.
- `emplace_back([&actor, &deadline] { ... })` — создаём поток, в котором лямбда крутит **свой актор** в цикле.
- `[&actor, &deadline]` — захват по ссылке. `actor` и `deadline` живут до конца `main` (вектор `threads` разрушится раньше `actor` и `deadline`), поэтому ссылки безопасны.
- Главный поток **ждёт** в цикле по 20 мс, проверяя `g_stop` и `deadline`. Он **не делает ходов** — только ждёт.
- По выходу из `if` — вектор `threads` **разрушается**, `jthread` деструкторы **делают join**, все потоки завершены.

**Почему главный поток не просто `sleep(seconds)`:** если нажали Ctrl+C — надо **сразу** выйти, а не ждать конца. `sleep_for(20ms)` + проверка `g_stop` — быстрая реакция.

## 10.8. Конец замера

```
auto t1 = std::chrono::steady_clock::now();
auto opsAfter = bank.GetOperationsCount();
auto elapsed = std::chrono::duration<double>(t1 - t0).count
();
auto totalOps = opsAfter - opsBefore;
```

- `t1 - t0` — **фактическое** время работы потоков (не `args.seconds`).
- `duration<double>` — секунды как `double` (например, `3.02`).
- `opsAfter - opsBefore` — **сколько операций сделано** за это время.

## 10.9. Проверка инвариантов

```
Money sumWallets = directorWallet.Get() + fatherWallet.Get()
+ motherWallet.Get()
                 + child1Wallet.Get() + child2Wallet.Get() +
sellerWallet.Get();
Money cash = bank.GetCash();
Money sumAccounts = bank.GetAccountBalance(directorAcc)
                  + bank.GetAccountBalance(fatherAcc)
                  + bank.GetAccountBalance(sellerAcc);

bool ok1 = (sumWallets == cash);
bool ok2 = (cash + sumAccounts == TOTAL);
```

- `sumWallets` — сумма всех кошельков.
- `cash` — наличные в обороте банка.
- `sumAccounts` — сумма всех балансов на счетах.

**Инвариант 1:** `Σ кошельки == cash` (у банка ровно столько наличных, сколько «на руках»).
**Инвариант 2:** `cash + Σ счета == TOTAL` (деньги не появились и не исчезли).

**Проверяем ПОСЛЕ** `join` всех потоков (или после выхода из `single`-цикла). Пока потоки бегут — состояние **промежуточное**, деньги могут быть «в воздухе» между `Take` и `Put`. После — всё стабильно.

### 10.10. Вывод и exit-код

```
std::cout << "mode: " << args.mode << "\n";
...
return (ok1 && ok2) ? 0 : 1;
```

**Exit-код 0** — если оба инварианта сходятся. **1** — если нет. `ctest` смотрит exit-код: если 1 — `Failed`.

---

# Часть 2. Теория по `test_bank.cpp`

## Что это

**Юнит-тесты** для `Bank` и `Wallet`. Не использует никаких фреймворков (Google Test, Catch2) — **самописный макрос** `CHECK`. Это допустимо: задание п.10 говорит «на любом фреймворке», но самописный — тоже вариант.

## Макрос `CHECK`

```
#define CHECK(cond) do { \
    if (!(cond)) { \
        std::cerr << "FAIL: " << #cond << " (line " << __LINE
__ << ")\n"; \
        std::exit(1); \
    } \
} while (0)
```

**Что делает:** если `cond` ложно — печатает провал и **выходит с кодом 1**.

**Тонкости:**

1. `#cond` — препроцессор превращает выражение в строку. `CHECK(b.GetCash() == 50)` → `"b.GetCash() == 50"`. Так при провале видно **что именно** не выполнилось.
2. `__LINE__` — встроенный макрос, номер строки. При провале видно, **где**.

3. `std::exit(1)` — завершает программу с кодом 1 (тест-файл **прерывается**, дальше не идёт).
4. `do { ... } while (0)` — трюк: макрос работает как **одна инструкция**. Без него `if (x) CHECK(y); else ...` сломалось бы (макрос из нескольких строк разрывает `if/else`).

# Тесты — по группам

**Успешные операции `Bank`:**

- `test_open` — `OpenAccount`, баланс 0.
- `test_deposit` — `DepositMoney`, баланс вырос, cash упал.
- `test_withdraw` — `WithdrawMoney`, баланс упал, cash вырос.

**Ошибки `Bank`:**

- `test_negative_deposit` — `DepositMoney(-1)` → `out_of_range`.
- `test_negative_withdraw` — `WithdrawMoney(-1)` → `out_of_range`.
- `test_negative_try_withdraw` — `TryWithdrawMoney(-1)` → `out_of_range`.
- `test_negative_initial_cash` — `Bank(-10)` → `out_of_range`.
- `test_no_account_deposit` — `DepositMoney(999999, 10)` → `BankOperationError`.
- `test_no_account_withdraw` — `WithdrawMoney(999999, 10)` → `BankOperationError`.
- `test_no_account_try_withdraw` — `TryWithdrawMoney(999999, 10)` → `BankOperationError`.
- `test_no_account_get_balance` — `GetAccountBalance(999999)` → `BankOperationError`.
- `test_no_account_close` — `CloseAccount(999999)` → `BankOperationError`.
- `test_not_enough_withdraw` — снять больше, чем есть → `BankOperationError`.
- `test_not_enough_cash_deposit` — положить больше наличных, чем есть в банке → `BankOperationError`.

**`Bank` — специальные:**

- `test_try_withdraw` — `TryWithdrawMoney` возвращает `false` при нехватке.
- `test_close` — `CloseAccount` возвращает остаток, счёта больше нет.

- `test_ops_count` — 2 вызова → +2.
- `test_failed_ops_counted` — провал исключения → +1.

`Wallet`:

- `test_wallet_take_put` — Take/Put.
- `test_wallet_negative` — отрицательные суммы → `out_of_range`.
- `test_wallet_transfer` — успех / не хватает / self / negative.
- `test_wallet_concurrent_transfer` — 4 потока × 10000 переводов, сумма сохраняется.

**Многопоточный:**

- `test_multithreaded` — 8 × 100 000 × 2 = 1 600 000 операций, balance и cash неизменны.

## Провал — это `std::exit(1)`, а не исключение

`CHECK` **не бросает исключение** — он **завершает программу**. Значит, тесты **последовательные**: один провал = всё завершилось. Это **нормально** для юнит-тестов (не хотим продолжать после провала).

## Где искать ошибку при провале

Если `test_multithreaded` упадёт с `FAIL: opsAfter - opsBefore == expected (line 234)` — ты сразу знаешь: строка 234, проверка не прошла.

---

## Дальше — код с огромным количеством комментариев

Сейчас пришлю **два файла** с построчными комментариями.

`main.cpp` — тот, что ты прислал, с комментариями на **каждый блок и каждую тонкость**.

`test_bank.cpp` — с комментариями на макрос `CHECK`, каждую группу тестов и тонкости.

Оба — в следующем сообщении, чтобы не разрывать теорию.

Если хочешь **сначала один файл** (например, `main.cpp` подробнее) — скажи, пришлю только его.

## Ключевые тонкости, на которые стоит обратить внимание

### В `main.cpp`

1. `[&]` в лямбдах — захват всего по ссылке. Безопасно, потому что все переменные живут дольше лямбд.
2. `argv[++i]` — сначала `++i`, потом `argv[i]`. Так «съедаем» следующий аргумент.
3. `std::stoi` **с** `&pos` — единственный способ отличить `"5"` от `"5hello"`. `"5hello"` даст `pos=1`, а длина строки — 6. Не совпало → отказ.
4. `std::function<void()>` — стирает тип лямбды, чтобы хранить все 6 в одном `vector<Actor>`.
5. `elapsed` через `steady_clock`, а не `args.seconds`. Если нажали Ctrl+C — время фактическое.
6. `return (ok1 && ok2) ? 0 : 1` — exit-код от инвариантов → ctest поймает.

### В `test_bank.cpp`

1. `CHECK(cond)` **с** `do {} while(0)` — макрос работает как одна инструкция. Без этого `if (x) CHECK(y); else ...` развалилось бы.
2. `#cond` — препроцессор делает из выражения строку, чтобы при провале видеть, **что** сломалось.
3. `std::exit(1)` **в** `CHECK` — тесты **прерываются** на первом провале. Это нормально для юнит-тестов.
4. **Порядок проверок в** `test_wallet_transfer`: сначала `amount < 0`, потом `self`. Поэтому `TransferTo(self, -10)` — это `out_of_range`, а не `false`.
5. `test_multithreaded` **— дословно по заданию п.11**: 8 × 100 000 × 2 = 1 600 000.